

# K.R. MANGALAM UNIVERSITY

Data Structures (ETCCDS202)

## Unit-4 Assignment: Non-Linear & Advanced Data Structures

|                           |                                       |
|---------------------------|---------------------------------------|
| <b>Name of the School</b> | School of Engineering & Technology    |
| <b>Program</b>            | B.Tech (AI and ML)                    |
| <b>Course Title</b>       | Data Structures                       |
| <b>Course Code</b>        | ETCCDS202                             |
| <b>Unit Title</b>         | Non-Linear & Advanced Data Structures |
| <b>Student Name</b>       | Akshit Rajput                         |
| <b>Roll Number</b>        | 2501730116                            |
| <b>Section</b>            | A                                     |
| <b>Semester</b>           | 2                                     |
| <b>Batch</b>              | 2025-29                               |
| <b>Submitted To</b>       | Mrs. Neetu Chauhan                    |
| <b>Submission Date</b>    |                                       |

## Introduction

This assignment focuses on implementing important non-linear and advanced data structures using Python. It includes a Binary Search Tree (BST), Graph traversal using BFS and DFS, and a Hash Table with separate chaining. The purpose is to understand how these structures help in efficient storage, searching, and data management, along with their real-world use in databases, routing, and caching.

## 1: Binary Search Tree (BST) Operations

### Why Inorder Traversal Prints a BST in Sorted Order

Inorder traversal follows the order Left → Root → Right. In a Binary Search Tree, all smaller values are stored in the left subtree and larger values are stored in the right subtree. Because of this property, visiting nodes in inorder always prints the elements in sorted (ascending) order.

### **Time Complexity (Simple Discussion)**

The average time complexity of insertion, search, and deletion in a BST is  $O(\log n)$  because each step reduces the search space by half. However, in the worst case, if the BST becomes skewed (like a linked list), the time complexity becomes  $O(n)$  since every node may need to be checked.

## **2: Graph Representation + BFS and DFS**

### **Difference Between BFS and DFS**

BFS (Breadth First Search) visits nodes level by level, exploring all neighboring nodes first before moving deeper. It uses a queue for traversal.

DFS (Depth First Search) goes deep into one branch first and then backtracks to explore other branches. It uses recursion or a stack.

### **Real-World Use Cases**

BFS is used for finding the shortest path in an unweighted graph, such as finding the minimum number of stops in a metro route.

DFS is used for cycle detection and path exploration, such as checking loops in networks or solving maze problems.

## **3: Hash Table with Separate Chaining (Collision Handling)**

### **What is a Collision and Why It Happens**

A collision occurs when two or more keys produce the same hash index in a hash table. This happens because the hash function may map different keys to the same location, especially when the table size is small.

### **Why Separate Chaining Helps**

Separate chaining handles collisions by storing multiple key-value pairs in the same index using a list (chain). Instead of overwriting data, all colliding elements are stored together, making insertion, search, and deletion possible even when collisions occur.

### **Time Complexity**

The average time complexity for insertion, search, and deletion is  $O(1)$  because data can usually be accessed directly using the hash index. In the worst case, if many collisions occur in one bucket, the complexity can become  $O(n)$ .

## **Conclusion**

This assignment helped in understanding the implementation and working of important data structures such as Binary Search Tree, Graph, and Hash Table. It showed how these structures are used for efficient searching, traversal, and data storage. Through this assignment, the practical importance, operations, and complexity of these data structures were understood along with their real-world applications.